A PRELIMINARY REPORT ON

AION: AUTONOMOUS INCIDENT OBSERVATION AND NAVIGATION

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)

SUBMITTED BY

ADITYA BHEKE	**Exam No : _____**
HARSHAL BHOGAWADE	**Exam No : _____**
MALHAR BHOI	**Exam No : _____**

Under the Guidance of
Dr. Yogesh Bahendwar



DEPARTMENT OF COMPUTER ENGINEERING

AJEENKYA DY PATIL SCHOOL OF ENGINEERING
CHARHOLI BUDRUK, LOHEGAON, PUNE 412105

SAVITRIBAI PHULE PUNE UNIVERSITY
2026-2027

AJEENKYA
DY Patil School of Engineering

AJEENKYA DY PATIL SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the Project Report Entitled

"AION: AUTONOMOUS INCIDENT OBSERVATION AND NAVIGATION"

Submitted by

ADITYA BHEKE	**Exam No : _____**
HARSHAL BHOGAWADE	**Exam No : _____**
MALHAR BHOI	**Exam No : _____**

are bonafide students of this institute and the work has been carried out by them under the supervision of **Dr. Yogesh Bahendwar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** in Computer Engineering.

(Dr. Yogesh Bahendwar) Guide Department of Computer Engineering	**Dr. Pankaj Agarkar** H.O.D Department of Computer Engineering
Signature of Internal Examiner:	Signature of External Examiner:

Dr. F. B. Sayyad
Principal
AJEENKYA DY PATIL SCHOOL OF ENGINEERING, Lohegaon, Pune – 412105

Place : Pune
Date :

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the preliminary project report on “AION: Autonomous Incident Observation and Navigation”.

We would like to take this opportunity to thank our Internal Guide **Dr. Yogesh Bahendwar** for giving us all the help and guidance we needed. We are really grateful to him for his kind support. His valuable suggestions were very helpful.

We are also grateful to **Dr. Pankaj Agarkar**, Head of Computer Engineering Department, ADYPSOE, Lohegaon, Pune and to our Project Co-ordinator **Dr. Amruta Chitari** for their indispensable support, suggestions and motivation during the entire course of the project.

We would also be grateful to our Principal **Dr. F. B. Sayyad** who encouraged us and created a healthy environment for all of us to learn in the best possible way.

We also thank all the staff members of our college and technicians for their help in making this project a successful one.

ADITYA BHEKE
HARSHAL BHOGAWADE
MALHAR BHOI

ABSTRACT

Resolving a production incident still requires engineers to search large volumes of logs, trace the failure to the code change that caused it, write and test a fix, and deploy it under time pressure, using several disconnected tools. Research in AIOps automates individual stages of this process, such as log parsing, anomaly detection, root-cause analysis and automated program repair, but the stages are usually studied in isolation, and letting a Large Language Model (LLM) change production code without checks is unsafe.

This report presents AION (Autonomous Incident Observation & Navigation), a closed-loop, human-gated platform that connects these stages. AION collects structured application logs, deduplicates them into error signatures and detects incidents as error spikes against a baseline. It ranks the commits that probably caused each incident with an explainable score based on git blame, function scope, dependency overlap and deployment timing, and asks an LLM, either the Claude API or a locally hosted model, for a diagnosis that must cite the evidence it is given. Candidate patches are generated on isolated branches under a code-enforced policy and validated by compilation, the existing tests, a staging deployment and a replay of the failing requests. Deployment is possible only through an explicit human approval that is bound to the exact validated commit, and every action is written to an append-only audit trail.

A working prototype of about 7.1 KLOC with 44 automated tests runs the complete loop. On four seeded scenarios it detected every incident with no false positive and ranked the true culprit first in all three faulty cases; a locally hosted 7B model named the culprit in two of three cases, and every incorrect AI patch was stopped by validation before reaching the approval gate. The report covers the problem definition, project plan, requirements specification, system design with UML and data-flow models, and a complexity analysis of the problem.

Keywords: AIOps, Incident Management, Log Analysis, Root-Cause Analysis, Large Language Models, Automated Program Repair, CI/CD, Human-in-the-Loop

TABLE OF CONTENTS

LIST OF ABBREVIATIONS.....v
LIST OF FIGURES.....vi
LIST OF TABLES.....vii
NOMENCLATURE.....viii

CHAPTER **TITLE** **PAGE NO.**

Sr. No.	Title of Chapter	Page No.
01	**Introduction**	**1**
1.1	Motivation of the Project	2
1.2	Literature Survey	3
02	**Problem Definition & Scope**	**7**
2.1	Problem Statement	7
2.2	Goal & Objective	7
2.3	Statement of Scope	8
2.4	Methodologies of Problem Solving and Efficiency Issues	9
03	**Project Plan**	**11**
3.1	Project Estimates	11
3.2	Risk Management w.r.t. NP-Hard Analysis	12
3.3	Project Schedule	14
04	**Software Requirements Specification**	**15**
4.1	Database Requirements	15
4.2	Software Requirements (Platform Choice)	16
4.3	Hardware Requirements	17
4.4	Analysis Models: SDLC Model to be Applied	17
05	**System Design**	**18**
5.1	System Architecture	18
5.2	Mathematical Model	19
5.3	Data Flow Diagrams	21
5.4	Entity Relationship Diagrams	23
5.5	UML Diagrams	24
06	**Other Specification**	**31**
6.1	Advantages	31
6.2	Limitations	31
6.3	Applications	31

Sr. No.	Title of Chapter	Page No.
07	**Conclusions & Future Work**	**32**
7.1	Conclusions	32
7.2	Future Work	32
	Appendix A: Problem Statement Feasibility Assessment	**33**
	Appendix B: Details of the Papers Referred	**35**
	Appendix C: Plagiarism Report	**37**
	References	**38**

LIST OF ABBREVIATIONS

ABBREVIATION	ILLUSTRATION
AIOps	Artificial Intelligence for IT Operations
API	Application Programming Interface
APR	Automated Program Repair
CI/CD	Continuous Integration / Continuous Deployment
CNF	Conjunctive Normal Form
COCOMO	Constructive Cost Model
DFD	Data Flow Diagram
FSM	Finite-State Machine
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
KLOC	Thousand Lines of Code
LLM	Large Language Model
LOC	Lines of Code
LSTM	Long Short-Term Memory
MTTR	Mean Time to Resolution
MVP	Minimum Viable Product
ORM	Object-Relational Mapper
RAM	Random Access Memory
RCA	Root-Cause Analysis
REST	Representational State Transfer
SAT	Boolean Satisfiability Problem
SDK	Software Development Kit
SDLC	Software Development Life Cycle
SHA	Secure Hash Algorithm (commit identifier)
SRE	Site Reliability Engineer
SSD	Solid-State Drive
UML	Unified Modeling Language
VRAM	Video Random Access Memory
WAL	Write-Ahead Logging

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE NO.
1.1	Conventional incident resolution compared with incident resolution using AION	2
2.1	Problem-solving methodology: four phases with a deterministic check after each	9
3.1	Project timeline (Gantt chart) for the academic year 2026–27	14
4.1	Incremental SDLC model applied to AION	17
5.1	System architecture of AION	18
5.2	DFD Level 0 (context diagram)	21
5.3	DFD Level 1	22
5.4	DFD Level 2 for process 5.0 (generate and validate patch)	23
5.5	Entity relationship diagram	24
5.6	Use case diagram	25
5.7	Class diagram	26
5.8	Sequence diagram for the resolution of one incident	27
5.9	Activity diagram	28
5.10	State machine diagram of the incident life cycle	29
5.11	Component diagram	30
5.12	Deployment diagram	30

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE NO.
1.1	Comparison of existing systems with AION	6
1.2	Stages covered by representative research work	6
2.1	Cost of each stage	10
3.1	Size of the AION prototype	11
3.2	Cost estimate	12
3.3	Risk register	12
4.1	Functional requirements	15
4.2	Database tables	15
4.3	Software requirements and platform choice	16
4.4	Hardware requirements	17
7.1	Preliminary evaluation results	32
A.1	Complexity class of each sub-problem	34

NOMENCLATURE

SYMBOL	DESCRIPTION
S	The system, S = { s, e, X, Y, F, DD, NDD, Sc, Fc }
X, Y, F	Sets of inputs, outputs and functions
L	Stream of log events; $l \in L$ is one event
C, D	Sets of recent commits and deployments
$\tau(l)$	Template of event l (variable values masked)
$s(l)$	Signature of event l
h	Hash function used to compute signatures
W, B	Detection window (60 s) and baseline period (30 min)
$c_t(s)$	Occurrences of signature s in the current window
$b(s)$	Average occurrences of s per window over the baseline
m, ρ	Minimum count (5) and spike ratio (3)
$f_k(c)$	Value (0 or 1) of correlation signal k for commit c
w_k	Weight of correlation signal k
$\lambda(c)$	Damping factor for code already running before the errors (0.5 or 1)
S(c)	Suspicion score of commit c
c*	Top-ranked suspect commit
E, R	Evidence pack and root-cause report
$\kappa(R)$	Confidence of report R
P	Candidate patch
V(P)	Validation result of patch P (1 = passed)
A	Human approval decision
sha(·)	Commit identifier of a patch or approval
N	Maximum number of AI patch attempts (2)
$M = (Q, \Sigma, \delta, q_0, F)$	Finite automaton of the incident life cycle
E (COCOMO)	Effort in person-months; D: duration in months

CHAPTER 1
INTRODUCTION

Modern software services run continuously and change often. Every release can introduce a fault, and when a fault reaches production it shows up as a stream of errors in the application logs, failed requests and unhappy users. The time between the first error and a verified fix, the mean time to resolution (MTTR), is one of the most closely watched measures of operational quality, because every minute of an outage has a direct cost for the business and its customers [1], [2].

In most teams the resolution of an incident is still a manual process. An on-call engineer is alerted, searches through thousands of log lines, decides which recent change caused the failure, writes a fix, tests it and finally pushes it through the deployment pipeline. Each of these steps uses a different tool, and each depends on the experience of the engineer who happens to be on call. Consider a typical case: an online store deploys release v1.4.0, which contains a coupon feature that makes the coupon lookup return `None` for expired codes. A few hours later customers who enter an old coupon receive HTTP 500 errors and the logs fill with `TypeError: 'NoneType' object is not subscriptable`. To resolve it, an engineer must notice that the error is new and not background noise, find the commit that introduced it, understand why it fails, write and test a fix, and deploy it. Every step is slow and error-prone when done by hand under time pressure.

Research in AIOps, the application of artificial intelligence to IT operations, has automated many of these steps separately: log parsers turn raw logs into templates, anomaly detectors flag unusual behaviour, root-cause analysis (RCA) methods localise faulty services, and Large Language Models (LLMs) can now explain failures and generate code fixes [2], [3]. However, these stages are rarely connected, and letting an LLM change production code without checks is unsafe, because a plausible but wrong fix can make an incident worse.

This project, **AION (Autonomous Incident Observation & Navigation)**, addresses this problem with a closed-loop, human-gated platform. AION collects structured application logs, groups repeated errors into signatures, detects incidents as error spikes, correlates each incident with the Git commits and deployments that preceded it, and

asks an LLM for an evidence-cited diagnosis and a candidate patch. Every patch is validated automatically on an isolated branch through compilation, the existing tests, a staging deployment and a replay of the failing requests. The guiding principle is that **AI may investigate, analyse, propose and validate, but only a human engineer can approve a production deployment**, and that approval is bound to the exact commit that passed validation. Figure 1.1 contrasts the conventional process with the AION workflow.

Chapter 2 defines the problem and scope, Chapter 3 the project plan, Chapter 4 the requirements and Chapter 5 the system design; Chapters 6 and 7 discuss the advantages, limitations and conclusions. Appendix A analyses the complexity of the problem and Appendix B summarises the papers referred.

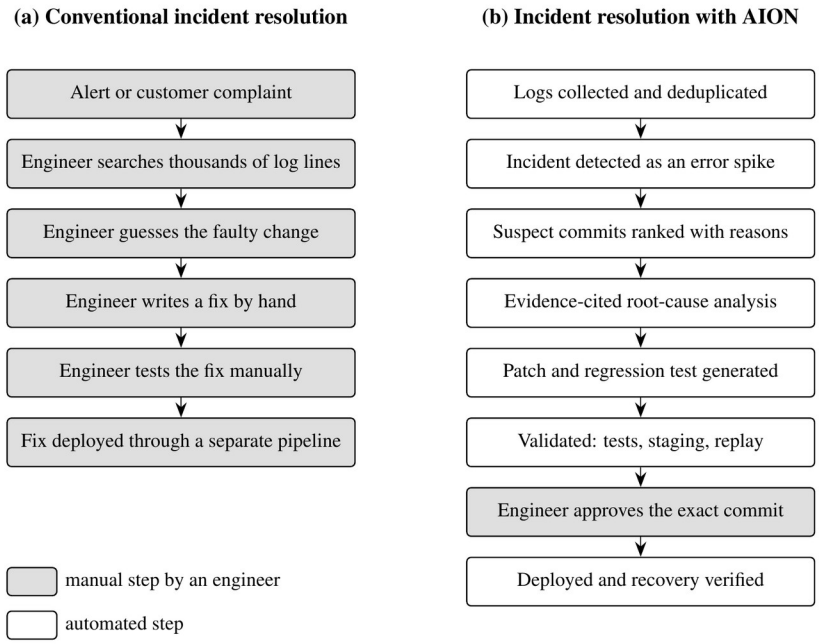


Figure 1.1: Conventional incident resolution compared with incident resolution using AION

Shaded boxes are manual steps performed by an engineer; white boxes are performed automatically by AION.

1.1 MOTIVATION OF THE PROJECT

Production incidents are among the most expensive moments in software engineering. A two-year study of incident management at Microsoft found that incidents are frequent, that their diagnosis depends heavily on expert knowledge, and that intelligent

automation is needed across the whole incident life cycle [1]. The motivation for AION comes from four observations.

- **Slow, manual resolution.** Locating the faulty change and writing a fix takes most of the resolution time and depends on experienced engineers being available.
- **Log and alert overload.** A single fault can produce thousands of identical log lines and many alerts. Surveys of log analysis show that grouping and filtering this volume is a research problem in its own right [4].
- **Fragmented tooling.** Observability platforms stop at detection, while code repair, testing and deployment are separate manual steps that use different tools.
- **Unsafe automation.** LLMs can recommend root causes and mitigation steps for real cloud incidents [5], [6], but they can also hallucinate causes and produce fixes that look right and are wrong. Recent work on LLM-based RCA explicitly calls for safety checks before any automated remediation [7].

AION is motivated by the gap between these two worlds: automating the slow and repetitive investigation work, while keeping deterministic validation and a human decision in front of every production change. The project also gives the team practical experience with technologies that are central to modern software engineering: observability, Git internals, LLM integration, automated testing and deployment pipelines.

1.2 LITERATURE SURVEY

The literature relevant to AION falls into four research areas: log analysis, AIOps and incident management, root-cause localisation, and automated program repair. Thirty-three publications were reviewed; the most important are summarised below and in Appendix B.

1.2.1 Log Parsing and Anomaly Detection

Surveys of automated log analysis cover the full pipeline from logging practice through parsing, anomaly detection, failure prediction and diagnosis; He *et al.* reviewed 158 publications spanning more than two decades [4]. Public datasets such as Loghub have enabled systematic benchmarking across distributed systems, supercomputers, operating systems and server applications [8]. *Log parsing* turns free-text messages into structured events: Drain parses logs online with a fixed-depth tree that groups messages into templates

[9], and LILAC uses LLMs with an adaptive parsing cache, improving accuracy while reducing the number of model queries [10].

Anomaly detection identifies abnormal log behaviour. DeepLog models normal log-key sequences with an LSTM [11], LogAnomaly detects sequential and quantitative anomalies [12], NeuralLog avoids explicit parsing by embedding raw messages and classifying them with a Transformer [13], and LogBERT and LAnoBERT apply self-supervised BERT models trained on normal logs [14], [15]. LLM-based log analysis is reviewed in [16], and a locally deployed LogBERT model is reported in [17]. These methods establish that something is wrong; they do not identify the responsible change or repair it. AION therefore uses a lightweight, deterministic detector (template masking and spike detection) whose output is easy to explain and audit.

1.2.2 AIOps and Incident Management

Notaro *et al.* organise AIOps failure-management methods into prevention, detection, diagnosis and remediation [2], and Zhang *et al.* survey how LLMs are changing these tasks, noting that the limits of LLMs in AIOps and the evaluation of LLM-based methods are not yet well understood [3]. Ahmed *et al.* evaluated GPT-3.x models on more than 40,000 incidents at Microsoft for recommending root causes and mitigation steps [5], and later work performs automatic RCA for cloud incidents with LLMs [6] and explores LLM-based agents for RCA [18]. These studies show that LLMs can assist engineers, but they work from incident text rather than from code and deployment history, and they do not produce or verify code changes.

1.2.3 Root-Cause Localisation and Change Analysis

Service-level RCA methods such as MicroRCA [19] and AutoMAP [20] localise faulty microservices from metrics and dependency graphs, and Soldani and Brogi survey anomaly detection and RCA in multi-service applications [21]. Wittkopp *et al.* progress from anomaly detection towards automated log labelling and RCA [22]. For change analysis, the SZZ approach of Śliwerski *et al.* links version history to bug reports to identify fix-inducing changes after the fact [23]. AION applies a related intuition online: when an error spike follows a deployment, the commits that touched the failing lines, functions or their dependencies, and that shipped in that deployment, are the strongest suspects.

1.2.4 Automated Program Repair and LLM Agents

Le Goues *et al.* review search-, semantics- and learning-based automated program repair (APR) [24]. AlphaRepair showed that pre-trained code models can repair bugs zero-shot [25], and Xia *et al.* found across nine models that larger pre-trained models outperform earlier APR tools [26]; Zhang *et al.* review the rapidly growing LLM-based APR literature [27]. RepairAgent gathers information, proposes fixes and validates them with tests, repairing 164 Defects4J bugs [28]; AutoCodeRover [29] and SWE-agent [30] resolve issues with LLM agents, evaluated on the 2,294 real GitHub issues of SWE-bench [31]. Kulsum *et al.* show that reasoning and patch-validation feedback improve LLM-generated vulnerability fixes [32]. These systems start from an issue or a failing test; none of them observes production, correlates deployments, reproduces failing production traffic or gates deployment on human approval.

1.2.5 Grounding and Human-in-the-Loop Control

Retrieval-augmented generation conditions a model on retrieved passages to make its output more specific and factual [33]. AION follows the same principle with a curated, bounded context in which every item has an identifier that the model must cite, so that claims can be checked mechanically. Human oversight is a recurring theme in incident-management research [1], [2]; AION makes it a structural property of the system, enforced by a state machine, rather than a recommendation.

1.2.6 Existing Systems and Research Gap

Widely used products cover parts of the workflow. The ELK stack (Elasticsearch, Logstash, Kibana) collects, indexes and visualises logs with dashboards and alerting, but root-cause analysis and fixing remain manual. Sentry groups application errors with their stack traces, links them to releases and suspect commits, and offers AI-assisted issue analysis and fix suggestions, while staging validation and deployment are left to the team's own pipeline. PagerDuty provides alerting, on-call scheduling, incident coordination and event grouping, but finding and fixing the faulty change is left to engineers. Table 1.1 compares these systems with AION, and Table 1.2 summarises the stages covered by representative research work.

Table 1.1: Comparison of existing systems with AION

System	Log analysis and detection	Suspect change identified	Code fix proposed	Fix validated (tests, staging, replay)	Human-gated deployment
ELK Stack	Yes	No	No	No	No
Sentry	Yes	Yes	Yes	No	No
PagerDuty	Alerting and grouping	No	No	No	No
AION (proposed)	Yes	Yes	Yes	Yes	Yes

Table 1.2: Stages covered by representative research work

Work	Detect	Correlate	RCA	Repair	Validate	Human gate
DeepLog, NeuralLog [11], [13]	Yes	–	–	–	–	–
MicroRCA, AutoMAP [19], [20]	Partial	–	Yes	–	–	–
LLM incident RCA [5], [6]	–	–	Yes	–	–	–
SZZ [23]	–	Yes	–	–	–	–
RepairAgent, AutoCodeRover [28], [29]	–	–	Partial	Yes	Yes	–
AION (this work)	Yes	Yes	Yes	Yes	Yes	Yes

The survey shows a progression from log monitoring to anomaly detection, root-cause analysis and LLM-based program repair, but each stage is usually studied and built separately, and validating and safely deploying a fix remain manual. The research gap addressed by AION is the safe, auditable composition of all these stages into one closed loop, with deterministic checks around every AI output and a human decision before production.

CHAPTER 2
PROBLEM DEFINITION & SCOPE

2.1 PROBLEM STATEMENT

To design and develop an AI-powered incident observation and remediation platform that collects and analyses production application logs, detects and deduplicates incidents, correlates them with Git commits and deployment history, performs evidence-based root-cause analysis using Large Language Models, generates candidate code patches that are validated through automated testing and staging, and deploys a fix to production only after explicit human approval of the exact validated commit.

The inputs to the system are the structured log stream of a monitored service, the Git history of its repository and its deployment records. For every incident the system must produce (i) a ranked list of suspect commits with an explanation for each rank, (ii) a diagnosis that states the root cause and cites the evidence, (iii) a candidate patch on an isolated branch together with a validation report, and (iv) a production deployment of that patch, performed only after a human decision. The formal model of these inputs and outputs is given in Section 5.2.

2.2 GOAL & OBJECTIVE

The goal of the project is to reduce the time and manual effort needed to resolve production incidents, without reducing the safety and accountability of production changes. The objectives are:

1. To collect and normalise production application logs from file and HTTP sources.
2. To group repeated errors into unique signatures using template masking and error fingerprints, and to detect incidents automatically as error spikes against a historical baseline.
3. To correlate each incident with Git commits and deployment history and rank the probable culprit commits with an explanation for every score.
4. To perform evidence-based root-cause analysis using LLMs, with grounding checks that reject claims not supported by evidence.

5. To generate candidate code patches and regression tests as reviewable diffs on isolated Git branches.
6. To validate every patch through compilation, the existing unit tests, a staging deployment and a replay of the failing requests.
7. To enforce a human approval gate, bound to the exact validated commit, before production deployment, and to verify the recovery of the service after deployment.
8. To keep a complete, append-only audit trail and to support both a cloud LLM (Claude API) and local LLMs, so that sensitive logs can stay on premises.
9. To evaluate the system on realistic bug scenarios with known root causes.

2.3 STATEMENT OF SCOPE

2.3.1 In Scope

- **Log ingestion:** collection and normalisation of structured JSON logs through file tailing and an HTTP API.
- **Incident detection:** deduplication of errors into signatures and detection of error spikes against a baseline.
- **Change correlation:** ranking of suspect commits using Git history and deployment records.
- **Root-cause analysis:** LLM-based, evidence-cited diagnosis, with a clearly labelled deterministic fallback when no LLM is configured.
- **Patch generation and validation:** AI-generated code edits with regression tests, or a revert of the suspect commit, validated through tests, staging and request replay.
- **Human approval, deployment and audit:** an approve / reject workflow bound to the validated commit, GitOps-style deployment with verification, a web dashboard and an append-only audit trail.
- **Evaluation:** measurement of detection, correlation, diagnosis and patch outcomes on seeded bug scenarios.

2.3.2 Out of Scope

The first version supports Python services and Python stack traces, with staging and production running as local processes. Languages such as Java and Node.js, metrics- and trace-based detection, Kubernetes and ArgoCD deployment, integration with hosted CI

systems such as GitHub Actions, authentication with role-based approvers, a container sandbox for validation and automatic rollback are outside the scope of this version and are planned as future work (Chapter 7).

2.3.3 Constraints and Assumptions

- The monitored service writes structured JSON logs that include the level, message, timestamp and, for errors, the stack trace.
- The service source code is available in a Git repository, and each deployment is recorded with its version and commit.
- The service has an automated test suite and commands to run it and to start the service, defined by the operator.
- An engineer is available to review and approve fixes; AION never deploys without that decision.

2.4 METHODOLOGIES OF PROBLEM SOLVING AND EFFICIENCY ISSUES

2.4.1 Problem Decomposition

The problem is solved by decomposition into four phases, each made of two stages and each closed by a deterministic check (Figure 2.1). The output of every stage is stored before the next stage starts, so a failure in a later stage never loses earlier results and every decision can be reviewed.

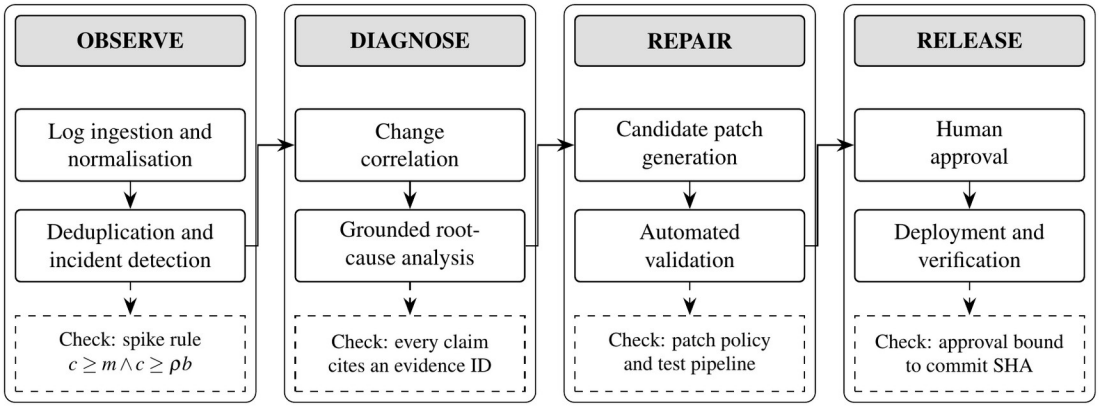


Figure 2.1: Problem-solving methodology: four phases with a deterministic check after each

Solid boxes are processing stages; dashed boxes are the deterministic checks that close each phase.

2.4.2 Techniques Used

- **Template masking and hashing** to collapse repeated errors into signatures.
- **Sliding-window spike detection** against a historical baseline to open incidents.
- **Explainable weighted scoring** of commits using `git blame`, function ranges from Python's `ast` module, file and dependency overlap and deployment timing.
- **Evidence packs with citation checking** to keep the LLM diagnosis grounded.
- **Constrained generate-validate-repair** with a code-enforced patch policy and a revert fallback.
- **A finite-state machine** whose only path to deployment passes through human approval.

2.4.3 Efficiency Issues

Table 2.1 lists the cost of each stage. The deterministic stages are linear or low-polynomial in the size of their inputs and finish in seconds; the dominant costs are LLM inference and test execution. In the evaluation, the deterministic path processed an incident in about 5 s, dominated by Git operations, the test suite and the staging start-up. With a locally hosted 7B model on a laptop GPU with 6 GB of memory, the end-to-end time rose to about 100 s per incident, with roughly 11–16k input and 1.2–1.3k output tokens per incident across one diagnosis and two patch attempts.

Table 2.1: Cost of each stage

Stage	Cost	Bounded by
Normalisation and signatures	$O(n \cdot l)$ for n events of length l	Batch size
Spike detection	O(s) indexed counts per batch, for s signatures	Window W and baseline B
Change correlation	$O(\|C\| \cdot L)$ for $\|C\|$ commits and L blamed lines	Number of recent commits
Root-cause analysis	One LLM call	Evidence pack size (12k-token context)
Patch generation	At most N LLM calls	N = 2 attempts (default)
Validation	Test suite, staging start, replay	180 s test timeout per command
Approval and deployment	O(1) checks and one fast-forward merge	Human decision

CHAPTER 3
PROJECT PLAN

3.1 PROJECT ESTIMATES

3.1.1 Size and Effort Estimation

The size of the software was measured directly from the working prototype rather than guessed. Table 3.1 gives the lines of code (LOC) of each part, which add up to about 7.1 thousand lines (KLOC).

Table 3.1: Size of the AION prototype

Part	Contents	LOC
Backend (Python)	API, log pipeline, correlation, AI layer, remediation, validation, deployment, audit	3,817
Dashboard (React)	Incident list, detail view, diff viewer, approval panel, audit and status pages	1,081
Demo and evaluation (Python)	Demo service, GitOps controller, bug scenarios, evaluation harness	1,478
Automated tests (Python)	Unit and end-to-end workflow tests	721
Total		**7,097**

The effort was estimated with the basic COCOMO model in organic mode, which suits a small team working on a well-understood problem with familiar tools. With KLOC = 7.1:

$$E = 2.4 \times (KLOC)^{1.05} = 2.4 \times 7.1^{1.05} \approx 18.8 \text{ person-months}$$

$$D = 2.5 \times E^{0.38} = 2.5 \times 18.8^{0.38} \approx 7.6 \text{ months}$$

$$P = E / D \approx 18.8 / 7.6 \approx 2.5 \text{ persons}$$

The estimate of about three persons over about eight months matches the team of three students and the academic year from July to April, which confirms that the plan in Section 3.3 is realistic.

3.1.2 Cost Estimation

AION was designed to run on the team's existing laptops with free and open-source software, so the hardware and software cost of the project is low (Table 3.2). The only

optional expense is usage of the Claude API; the system works without it, using a local model or the deterministic fallback.

Table 3.2: Cost estimate

Item	Details	Cost (₹)
Development laptops (3)	Core i5 / Ryzen 5, 16 GB RAM, 512 GB SSD; already owned	0
GPU for local LLM	NVIDIA RTX 3050, 6 GB VRAM, in one laptop; already owned	0
Software and tools	Python, FastAPI, SQLite, React, Git, pytest, Ollama, VS Code: open source or free	0
Internet connection	Existing broadband / college Wi-Fi	0
Claude API usage	Pay per use; budget for development and evaluation runs (estimate)	≈ 2,500
Total		**≈ 2,500**

3.2 RISK MANAGEMENT W.R.T. NP-HARD ANALYSIS

3.2.1 Risk Identification and Mitigation

Risks were identified for the product (what can go wrong when AION runs) and for the project (what can delay its completion). Table 3.3 lists each risk with its probability, impact and mitigation.

Table 3.3: Risk register

Risk	Prob.	Impact	Mitigation
LLM gives a wrong or hallucinated diagnosis	High	High	Evidence IDs and grounding checks; confidence capped for unsupported claims; human review
AI patch is incorrect	High	High	Validation pipeline; bounded repair loop; revert fallback; human approval
AI patch edits tests, CI or dependency files	Medium	High	Code-enforced patch policy rejects such edits
Wrong commit ranked first	Medium	Medium	Several independent signals with reasons; top suspects and diffs shown to the LLM and the engineer
False-positive incidents from noise	Medium	Medium	Minimum count plus spike ratio against a baseline
Sensitive data in logs sent to an external LLM	Medium	High	Local LLM option; redaction planned

Risk	Prob.	Impact	Mitigation
AI-generated code runs on the host during validation	Medium	High	Separate worktree and process with timeouts; container sandbox planned
Unauthorised approval	Medium	High	Server bound to localhost; authentication and roles planned
LLM unavailable or too costly	Medium	Medium	Provider-neutral layer; local model; deterministic fallback
Schedule slip	Low	Medium	Incremental model: a working system after the first increment

3.2.2 Risks Arising from Problem Complexity

Two sub-problems of AION are computationally hard, and this hardness is itself a project risk because an exact algorithm could run for an unbounded time. The analysis is given in Appendix A; its consequences for risk management are:

- **Patch synthesis.** Finding a program change that makes all tests pass is undecidable in general, and its bounded version (a fixed set of choices) is NP-complete by reduction from Boolean satisfiability. AION therefore never searches exhaustively: it asks the LLM for at most N candidates (N = 2 by default), limits each patch to 5 files and 200 changed lines, runs every command with a timeout, and falls back to reverting the top suspect.
- **Minimal explanation of an incident.** Choosing the smallest set of commits that explains all failing signatures is an instance of the Set Cover problem, which is NP-complete. AION avoids it by scoring every commit independently in polynomial time and ranking the scores; the engineer reviews the top candidates.
- **Verification stays polynomial.** Once a candidate exists, checking it (compilation, tests with timeouts, staging start-up, replay of a bounded set of requests) takes bounded time. The design therefore follows a "propose, then verify" structure: the hard search is delegated to a heuristic (the LLM), and every proposal is verified by polynomial-time, deterministic checks.

3.2.3 Risk Monitoring

Every automated and human action is written to the append-only audit trail, so the behaviour of the system can be reviewed after each incident. The evaluation harness is re-run after every change to the detection, correlation or patching logic; in the first evaluation round it exposed four weaknesses (indentation handling in AI edits, keyword noise, a

missing dependency signal and the absence of a rollback path), all of which were fixed generically and covered by unit tests.

3.3 PROJECT SCHEDULE

The project spans the academic year 2026–27. Semester I covers problem definition, literature survey, requirement analysis, design and a working prototype with a first evaluation; Semester II covers hardening, integration, extended evaluation and final documentation. Figure 3.1 shows the timeline. At the time of this report all Semester I tasks are complete: the literature survey of 33 publications, the requirement analysis, the design, the MVP pipeline, the local-LLM integration with its evaluation, and the synopsis and reviews. The Semester II tasks are planned.

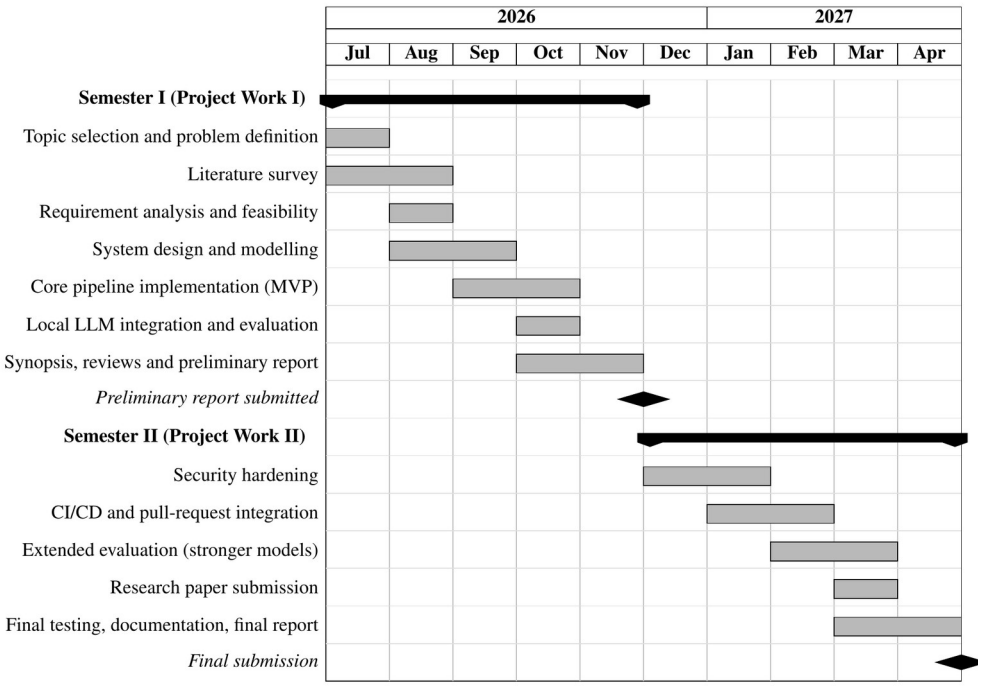


Figure 3.1: Project timeline (Gantt chart) for the academic year 2026–27

Grey bars are tasks; black bars group the tasks of each semester; diamonds are milestones.

CHAPTER 4
SOFTWARE REQUIREMENTS SPECIFICATION

This chapter specifies what AION must do and the platform it runs on. The functional requirements are listed in Table 4.1; each is traced to the design section that addresses it.

Table 4.1: Functional requirements

ID	Requirement	Addressed in
FR1	Ingest structured logs from files and over HTTP	Section 5.1
FR2	Collapse repeated errors into signatures	Section 5.2
FR3	Open an incident when the rate of an error spikes	Section 5.2
FR4	Rank suspect commits with explanations	Section 5.2
FR5	Diagnose the root cause with cited evidence	Section 5.1
FR6	Generate a candidate fix and a regression test	Section 5.3
FR7	Validate the fix against tests and replayed traffic	Section 5.3
FR8	Deploy only after approval of the validated commit; verify recovery	Section 5.5
FR9	Work without an LLM, and with a local model	Section 4.2
FR10	Record every automated and human action in an audit trail	Section 5.4

The main non-functional requirements are **safety**, **explainability**, **auditability**, **portability** (Windows and Linux, cloud or local LLM) and **responsiveness** of the dashboard during analysis.

4.1 DATABASE REQUIREMENTS

All state is stored in SQLite in write-ahead-logging (WAL) mode through the SQLAlchemy 2.0 ORM, so the complete history of an incident can be reconstructed; PostgreSQL is planned for multi-worker use. Table 4.2 lists the tables.

Table 4.2: Database tables

Table	Purpose and key fields
services	Monitored services: name, repository path, production branch, log path, test and run commands, health path
log_events	Normalised log records: timestamp, level, message, template, signature, exception, stack frames, request
log_signatures	One row per distinct template: signature, template, count, first and last occurrence

Table	Purpose and key fields
incidents	Detected incidents: title, status, severity, signature, detection reason, event count
deployments	Releases: environment, version, commit SHA, time, status, related incident
commit_suspects	Scored commits for an incident: SHA, author, message, files, score, reasons
rca_reports	Diagnoses: root cause, observations, inferences, suspected commit, confidence, warnings, evidence pack
patch_proposals	Candidate patches: attempt, strategy, edits, diff, branch, commit SHA, status
validation_runs	Validation results for a patch: status and step-by-step results
approvals	Human decisions: commit SHA, decision, approver, comment
audit_events	Append-only trail: time, actor, action, details

Two integrity rules are essential: deployment is refused if the approved commit SHA differs from the validated commit, and no API exists to update or delete audit events.

4.2 SOFTWARE REQUIREMENTS (PLATFORM CHOICE)

The platform was chosen for maturity and low cost (Table 4.3).

Table 4.3: Software requirements and platform choice

Layer	Choice	Reason
Language	Python 3.10+	Rich ecosystem for Git, testing and LLM SDKs; built-in `ast` module for code analysis
Web framework	FastAPI, Uvicorn, Pydantic	Typed REST API with validation and automatic OpenAPI documentation
Database	SQLite (WAL) + SQLAlchemy 2.0	Zero administration, transactional; can be replaced by PostgreSQL
Frontend	React.js + Vite	Component-based dashboard with a fast build
Version control	Git CLI	History, blame, worktrees, branches, reverts and fast-forward merges
Testing	pytest, httpx	Project tests, patch validation and request replay
Cloud LLM	Claude API (Anthropic SDK)	Strong code reasoning and structured outputs
Local LLM	Ollama with qwen2.5-coder:7b	Offline and private analysis through an OpenAI-compatible API
Environment	Windows / Linux, VS Code, Node.js 18+	Available to all team members at no cost

4.3 HARDWARE REQUIREMENTS

No dedicated server is needed; a GPU is needed only for a local LLM (Table 4.4).

Table 4.4: Hardware requirements

Component	Minimum	Recommended
Processor	Intel Core i5 / AMD Ryzen 5	Intel Core i5 / AMD Ryzen 5 or better
Memory	8 GB RAM	16 GB RAM (needed for a local LLM)
Storage	SSD with 20 GB free	SSD with 20 GB free or more
GPU	Not required	NVIDIA GPU with at least 6 GB VRAM for a local LLM
Network	Internet access for GitHub and the cloud LLM	Stable broadband
Display	1920 × 1080	1920 × 1080 or higher

4.4 ANALYSIS MODELS: SDLC MODEL TO BE APPLIED

The project follows the **incremental model** (Figure 4.1): the system is delivered in increments, each passing through requirements, design, implementation and testing. It was chosen because the main risks lie in the AI stages, which can only be judged by running them, because every increment yields a usable system for review, and because the evaluation of one increment sets the requirements of the next. Increment 1 delivered the closed-loop MVP, Increment 2 the local LLM and evaluation, and Increment 3 (Semester II) adds hardening and integration.

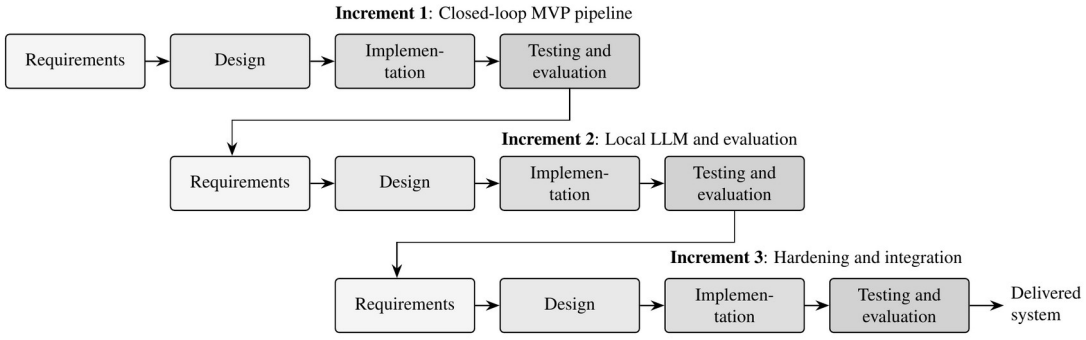


Figure 4.1: Incremental SDLC model applied to AION

Each increment passes through all four phases; darker shading indicates later phases.

CHAPTER 5
SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

AION is built as a modular monolith: a single backend process whose packages have clear boundaries and communicate through function calls and the database. Figure 5.1 shows the closed-loop architecture. A monitored production service emits structured JSON logs; AION processes them through eight stages; stages 1–6 run automatically in a background pipeline, stage 7 requires a human, and stage 8 runs only after approval. Every transition is appended to the audit trail.

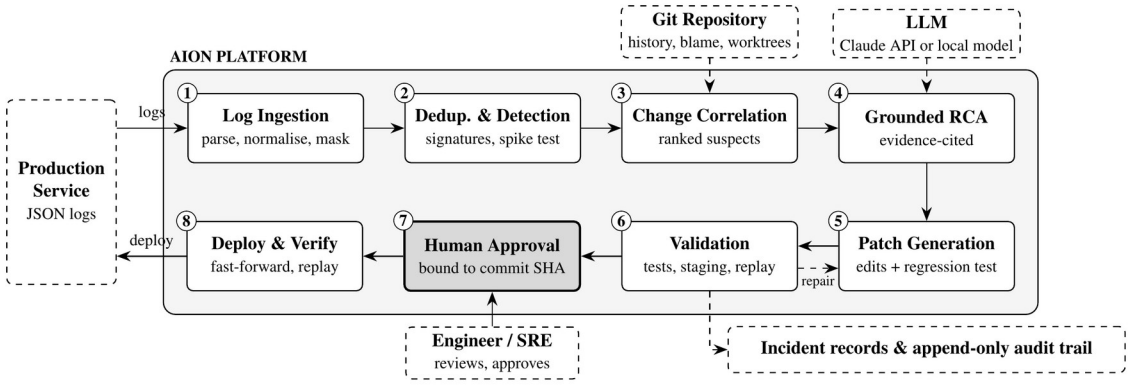


Figure 5.1: System architecture of AION

Numbered boxes are the eight stages; the shaded stage requires a human decision. Dashed boxes are external systems; the dashed arrow from validation back to patch generation is the bounded repair loop.

1. **Log ingestion:** file-tail collector and HTTP API; normalisation and stack-frame parsing.
2. **Deduplication and detection:** templates, signatures and spike detection against a baseline.
3. **Change correlation:** scoring of recent commits with a recorded reason for every signal.
4. **Grounded RCA:** an evidence pack with stable identifiers; claims checked against the evidence.
5. **Patch generation:** search/replace edits and a regression test on an isolated worktree and branch.

6. **Validation:** compilation, reproduction check, test suite, staging start-up and request replay.
7. **Human approval:** the engineer approves or rejects the exact validated commit.
8. **Deployment and verification:** pre-flight checks, fast-forward merge and verification on production.

The LLM layer is provider-neutral: a single interface is implemented for the Claude API and for any OpenAI-compatible server such as Ollama. When no model is configured, a clearly labelled deterministic analyser produces a diagnosis from the top-ranked suspect and proposes a revert, so the workflow never depends on the availability of an LLM.

5.2 MATHEMATICAL MODEL

The system is modelled as a set-theoretic system S. The symbols used below are defined in the Nomenclature.

$$S = \{ s, e, X, Y, F, DD, NDD, Sc, Fc \} \tag{5.1}$$

where s is the initial state (a monitored service with no open incident), e is the final state (incident resolved or rejected), X is the set of inputs, Y the set of outputs, F the set of functions, DD the deterministic data, NDD the non-deterministic data, Sc the success condition and Fc the failure conditions.

Inputs. $X = \{ L, C, D, \Theta \}$, where L is the stream of log events, C the set of recent commits, D the set of deployments and $\Theta = \{ W, B, m, \rho, w, \lambda, N \}$ the configuration. **Outputs.** $Y = \{ I, \hat{C}, R, P, V, A \}$: the incident, the ranked suspects, the diagnosis, the patch, the validation result and the approval. **Functions.** $F = \{ f_{norm}, f_{sig}, f_{det}, f_{corr}, f_{rca}, f_{patch}, f_{val}, f_{dep} \}$.

Normalisation and signatures. Each event $l \in L$ is normalised to a template $\tau(l)$ by replacing variable values with placeholders. Its signature is a hash h of the level, logger, template, exception type and the innermost three application frames:

$$s(l) = h(level(l), logger(l), \tau(l), type(l), frames(l)) \tag{5.2}$$

Detection. For a signature s at event time t, let $c_t(s)$ be the number of events with signature s in the window $(t - W, t]$, and b(s) the average number per window over the preceding baseline of length B. An incident is opened when

$$f_{det}(s, t) = 1 \iff c_t(s) \geq m \ \wedge \ c_t(s) \geq \rho \cdot b(s) \tag{5.3}$$

with defaults W = 60 s, B = 30 min, m = 5 and ρ = 3. A brand-new error (b(s) = 0) fires as soon as it reaches m occurrences, while long-standing low-rate noise does not.

Correlation. For each commit c ∈ C, binary signals $f_k(c) \in \{0, 1\}$ are computed and combined into an explainable suspicion score:

$$S(c) = \min(\ 1, \Sigma_k\ w_k \cdot f_k(c)\ ) \cdot \lambda(c) \tag{5.4}$$

$$c^* = \arg\max_{c \in C} S(c) \tag{5.5}$$

The weights are 0.40 (blame on the failing line), 0.20 (blame on the enclosing function), 0.20 (shipped in the deployment just before the errors), 0.15 (touched a file in the stack trace), 0.15 (touched a module imported by such a file) and 0.05 (error keywords in the commit message). λ(c) = 0.5 if the commit was already running before the errors began and 1 otherwise. S(c) is a ranking heuristic, not a probability.

Grounded diagnosis. The diagnosis $R = f_{rca}(E)$ is produced from the evidence pack E. Let κ(R) be its confidence. If any claim cites an unknown evidence identifier, κ(R) is capped at 0.60; if no verifiable observation is cited, it is capped at 0.30.

Validation and deployment. A patch P is offered for approval only if every blocking validation step passes; the reproduction check (the new test must fail on the old code) is recorded and reported as a warning when it does not hold:

$$V(P) = \text{syntax}(P) \wedge \text{tests}(P) \wedge \text{staging}(P) \wedge \text{replay}(P) \tag{5.6}$$

$$\text{deploy}(P) \iff V(P) = 1 \ \wedge \ A = \text{approve} \ \wedge \ \text{sha}(A) = \text{sha}(P) \tag{5.7}$$

Deterministic data (DD): thresholds, weights, policy limits (5 files, 200 changed lines, protected paths) and operator-defined commands. **Non-deterministic data (NDD):** LLM outputs, which vary between runs, and the arrival times of log events.

Success condition (Sc): the approved commit is running in production and the previously failing requests succeed, after which the incident moves to *resolved*. **Failure conditions (Fc):** validation fails after N attempts and the revert fallback (*validation_failed*), deployment verification fails (*deploy_failed*), a pipeline step raises an error (*error*), or the engineer rejects the fix (*rejected*).

5.3 DATA FLOW DIAGRAMS

The data-flow diagrams use the Yourdon–DeMarco notation: circles are processes, rectangles are external entities, open rectangles are data stores and arrows are data flows.

5.3.1 DFD Level 0

The context diagram (Figure 5.2) shows AION as a single process exchanging data with four external entities: the production service, the Git repository, the LLM provider and the engineer.

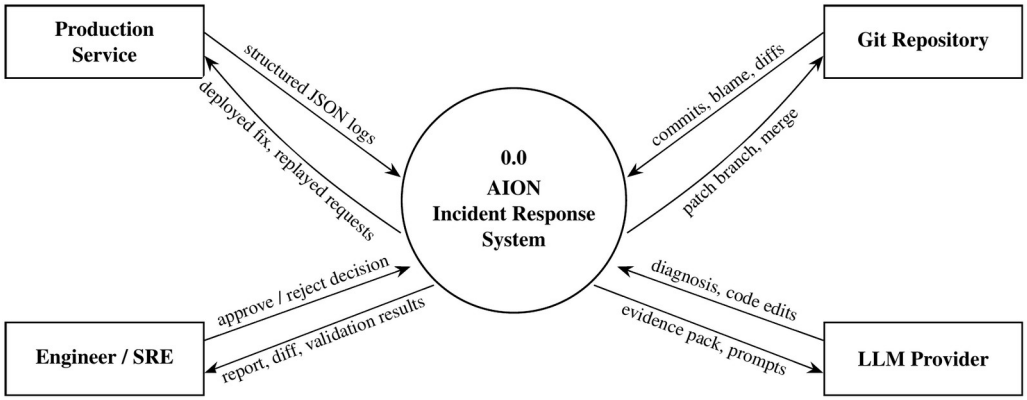


Figure 5.2: DFD Level 0 (context diagram)

5.3.2 DFD Level 1

Level 1 (Figure 5.3) decomposes the system into six processes. Each process writes its result to a data store that the next process reads, which is how the database preserves the full history of every incident.

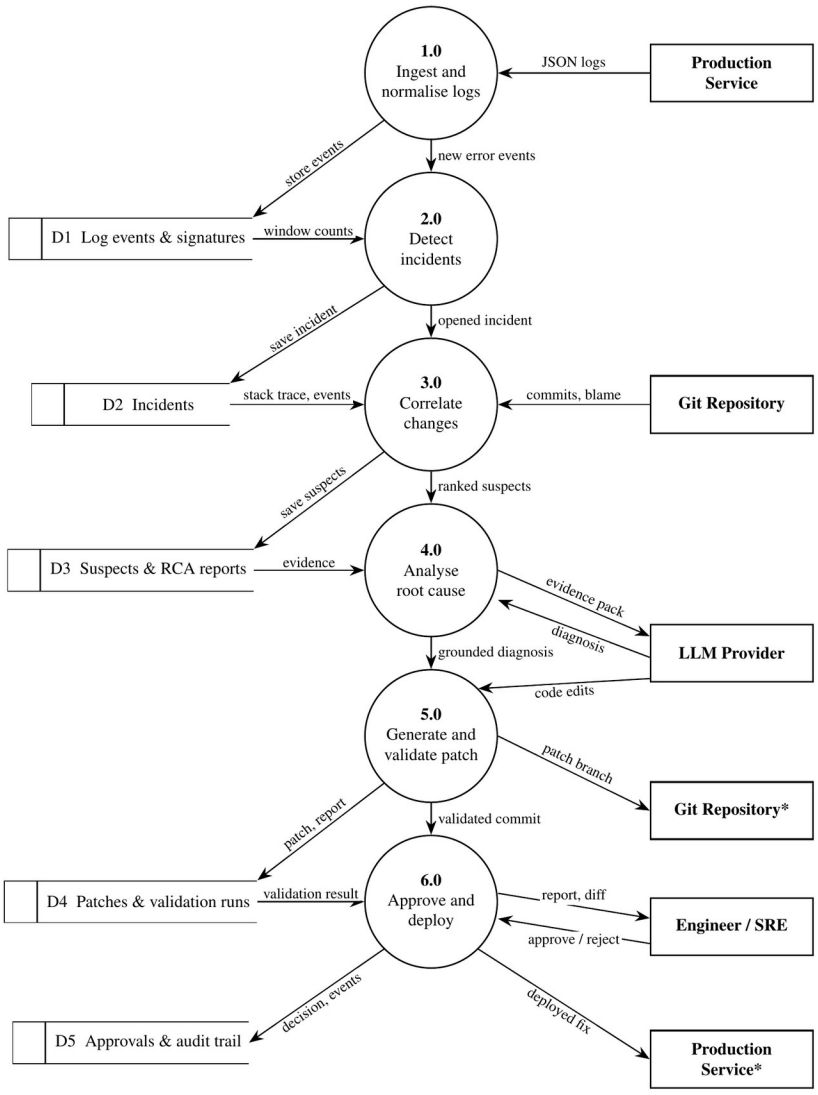


Figure 5.3: DFD Level 1

An asterisk marks an external entity drawn a second time to avoid crossing lines.

5.3.3 DFD Level 2

Level 2 (Figure 5.4) expands process 5.0, the generation and validation of a patch, including the bounded repair loop and the revert fallback.

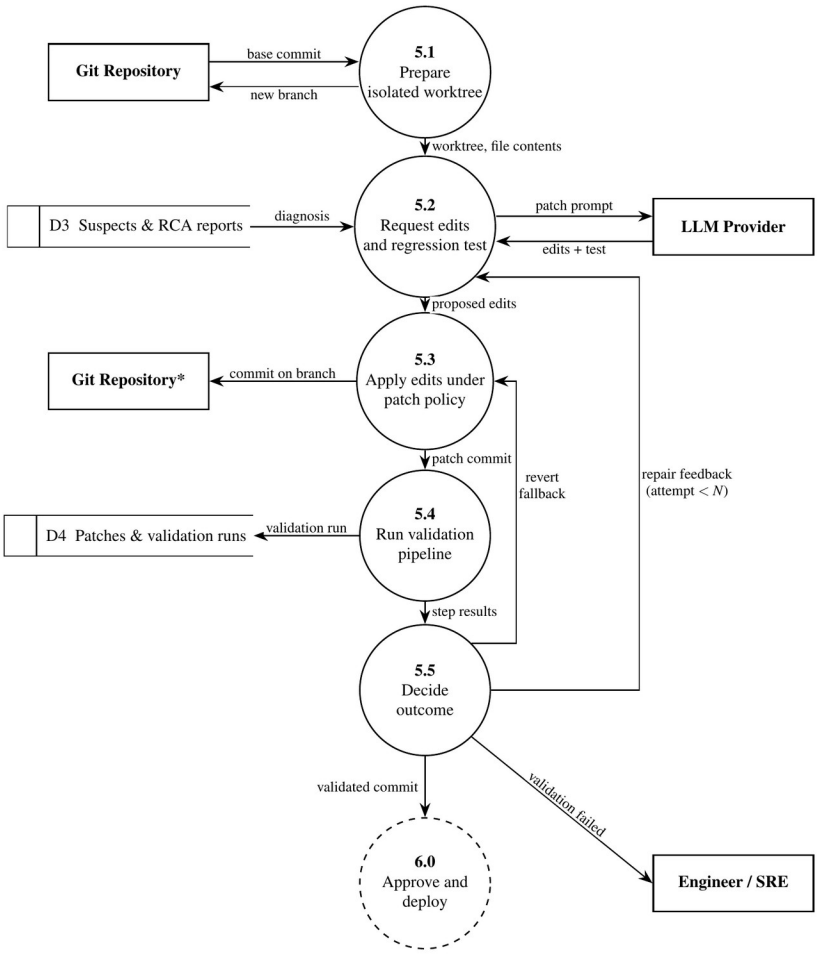


Figure 5.4: DFD Level 2 for process 5.0 (generate and validate patch)

The dashed circle is process 6.0 of Level 1. N is the maximum number of AI patch attempts (default 2).

5.4 ENTITY RELATIONSHIP DIAGRAMS

Figure 5.5 shows the entities, their key attributes and their relationships. Incident is the central entity: every analysis artefact (suspects, diagnoses, patches, validation runs, approvals and audit events) belongs to exactly one incident, so a reviewer can reconstruct the full history of an incident from the database alone.

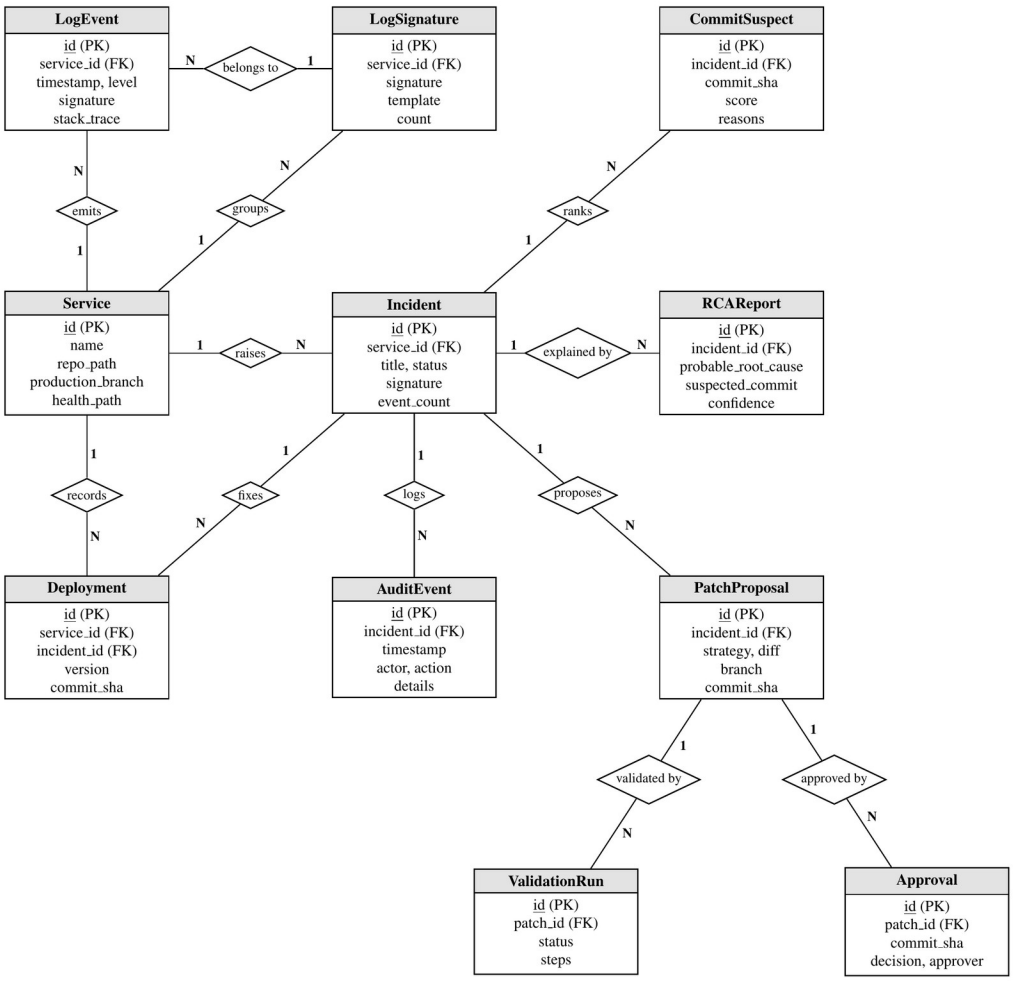


Figure 5.5: Entity relationship diagram

Rectangles are entities with their key attributes; diamonds are relationships; 1 and N give the cardinalities.

5.5 UML DIAGRAMS

5.5.1 Use Case Diagram

Figure 5.6 shows the actors and use cases. The engineer reviews incidents, approves or rejects fixes, triggers deployment and inspects the audit trail; the production service, the Git repository and the LLM provider are system actors that take part in the automated use cases.

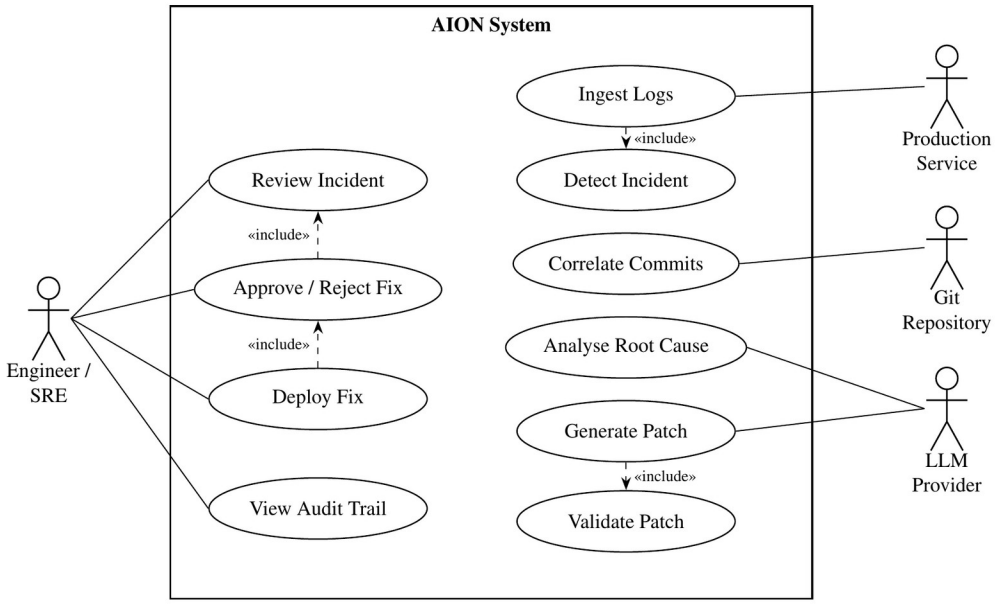


Figure 5.6: Use case diagram

5.5.2 Class Diagram

Figure 5.7 shows the main classes of the implementation. The `LLMProvider` interface is realised by `AnthropicProvider` and `OpenAICompatProvider`, so the rest of the system does not depend on a particular model vendor.

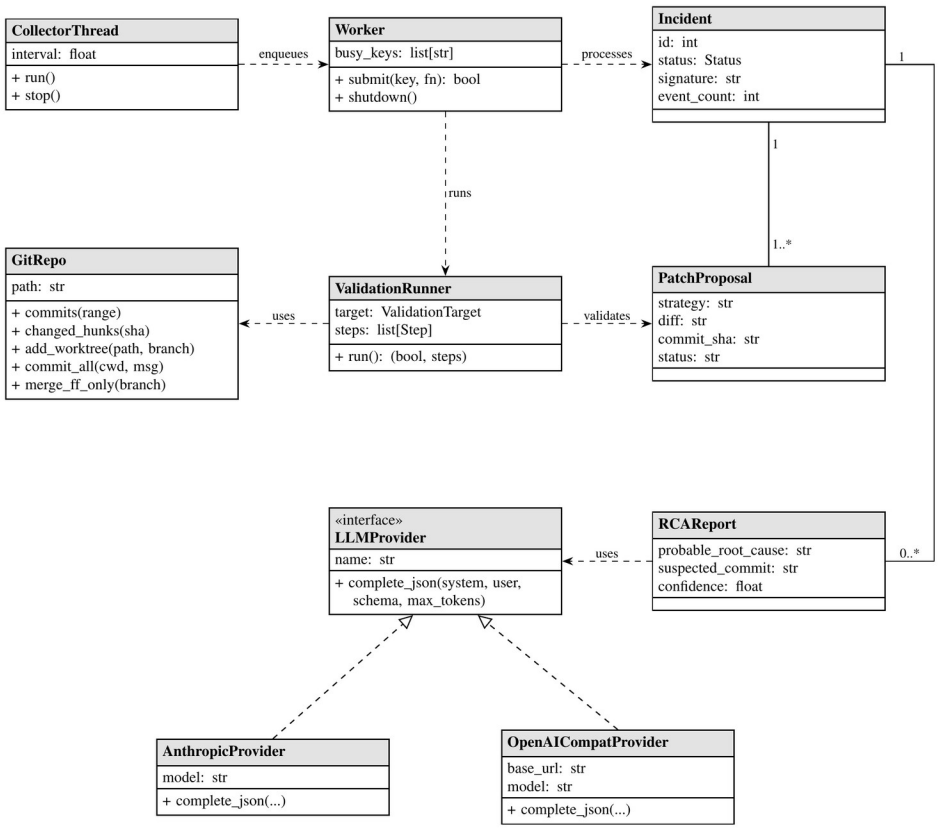


Figure 5.7: Class diagram

5.5.3 Sequence Diagram

Figure 5.8 shows the interactions for one incident, from the arrival of error logs to the verified deployment of the approved fix.

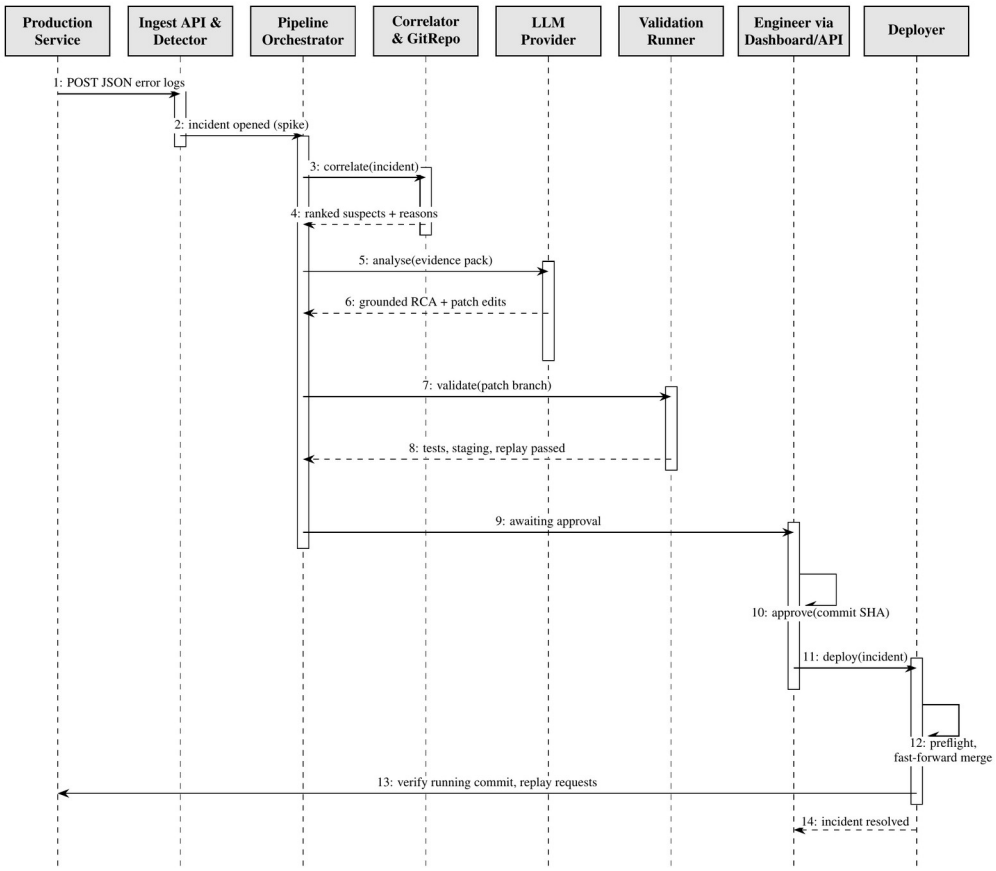


Figure 5.8: Sequence diagram for the resolution of one incident

Solid arrows are calls; dashed arrows are returns.

5.5.4 Activity Diagram

Figure 5.9 shows the control flow, including the bounded repair loop, the revert fallback and the human decision points.

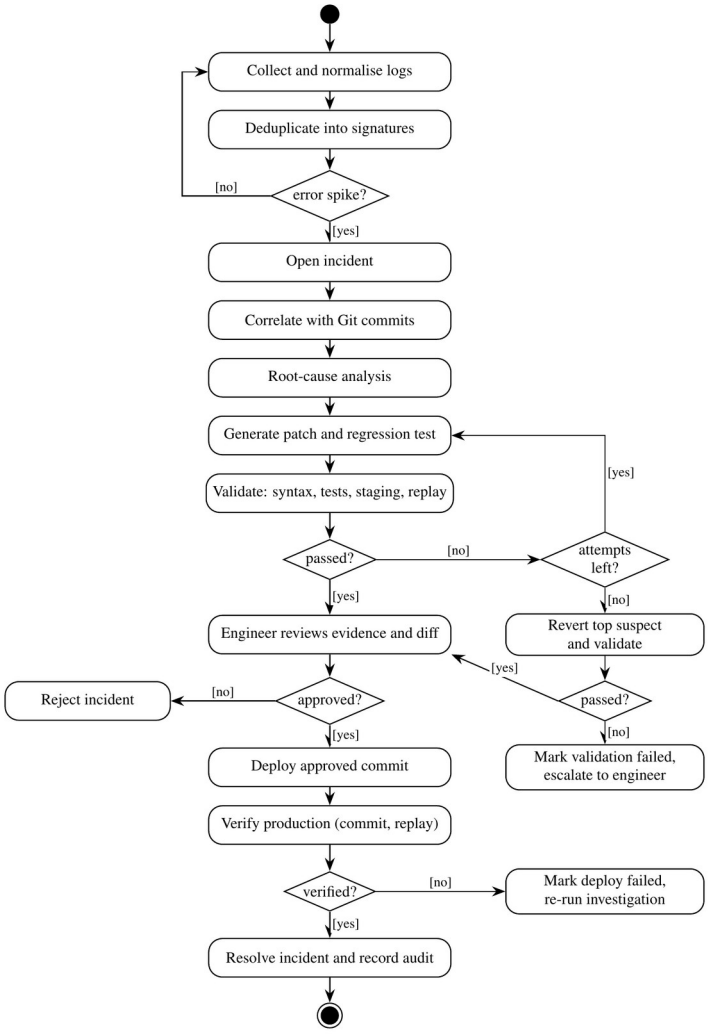


Figure 5.9: Activity diagram

5.5.5 State Machine Diagram

The incident life cycle is an explicit finite-state machine (Figure 5.10). Every status change is checked against the list of allowed transitions; there is no transition from *awaiting_approval* to *deploying*, so the only way to production passes through *approved*, which only the human approval API can set.

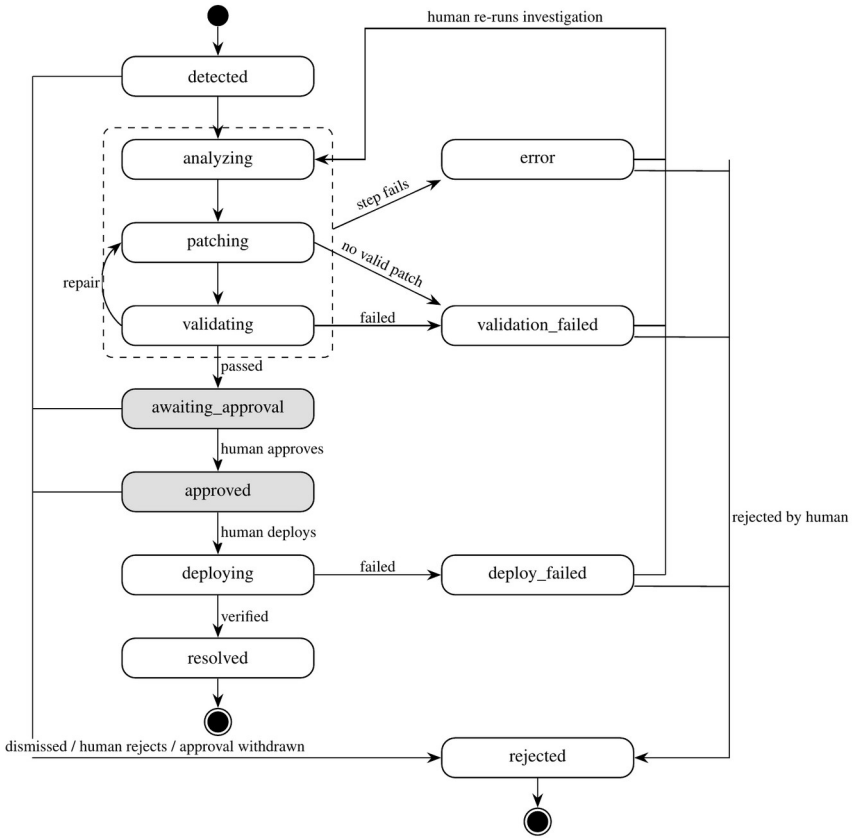


Figure 5.10: State machine diagram of the incident life cycle

Shaded states require a human decision. The dashed region groups the automated states from which a failing step leads to error.

5.5.6 Component Diagram

Figure 5.11 shows the components of the backend and their dependencies on external systems.

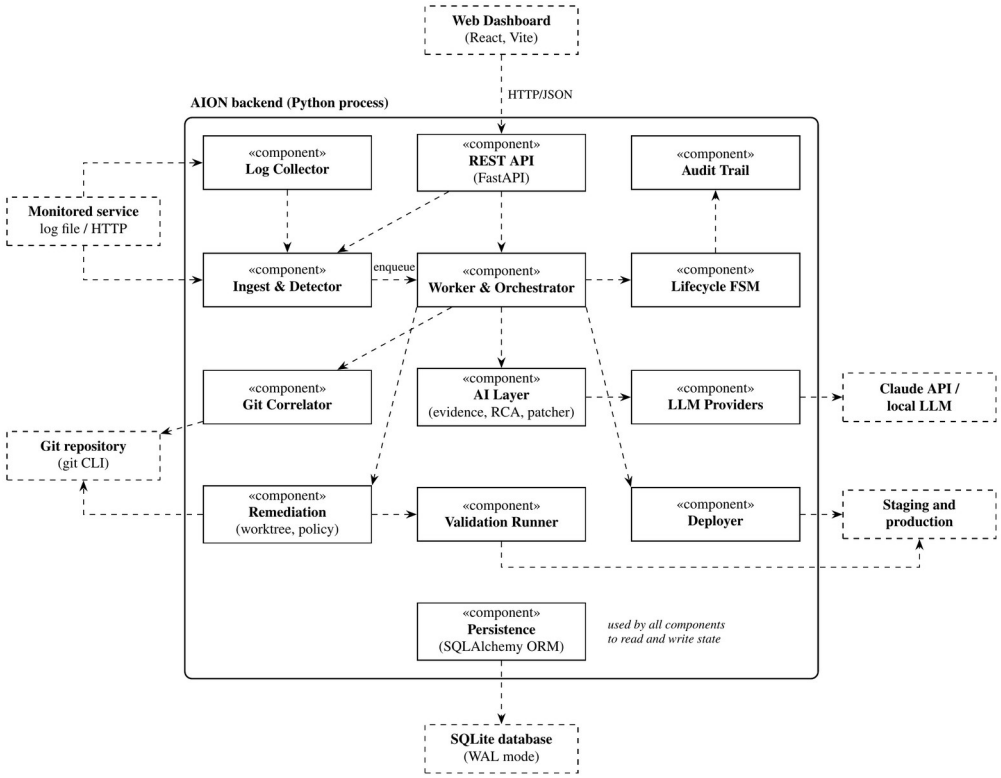


Figure 5.11: Component diagram

Dashed arrows are dependencies; dashed boxes are external systems.

5.5.7 Deployment Diagram

Figure 5.12 shows the deployment of the prototype. AION, the monitored service and its staging copy run on one laptop; the local LLM runs on a machine with a GPU, which may be the same laptop, and the Claude API is reached over HTTPS.

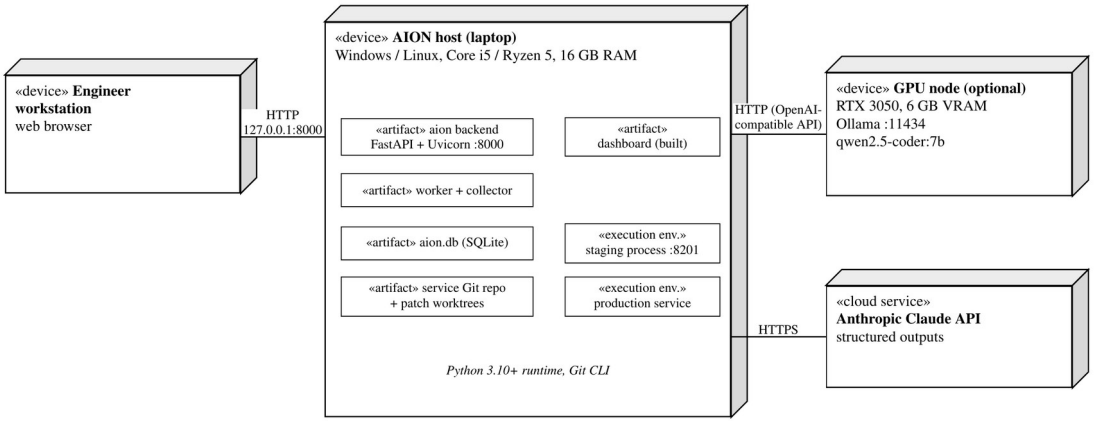


Figure 5.12: Deployment diagram

CHAPTER 6
OTHER SPECIFICATION

6.1 ADVANTAGES

- **Faster resolution:** an injected fault reached the approval gate about 4 s after the first errors in a live demonstration.
- **Safe by construction:** no production change is possible without a human approval bound to the validated commit.
- **Explainable and auditable:** suspect scores carry reasons, diagnoses cite evidence, and every action is recorded in an append-only trail.
- **Model-neutral and private:** works with the Claude API, local models or no model at all, so sensitive logs can stay on premises.
- **Robust and low cost:** wrong patches are caught by validation; runs on existing laptops with free software.

6.2 LIMITATIONS

- Only Python services are supported; detection uses logs only, with fixed thresholds and hand-tuned weights.
- Staging and production are local processes, and request replay covers GET and HEAD requests only.
- AI-generated code runs on the host during validation, without a container sandbox.
- There is no authentication yet, so the server binds to the local machine only.
- The evaluation is small (four scenarios on one service).

6.3 APPLICATIONS

- **Site reliability engineering and on-call support:** first-line investigation of incidents in web services and APIs.
- **DevOps teams and start-ups** without a dedicated operations team.
- **Regulated environments** that need an audit trail, human sign-off and local models for sensitive data.

CHAPTER 7
CONCLUSIONS & FUTURE WORK

7.1 CONCLUSIONS

This report presented AION, a closed-loop, human-gated platform that connects incident detection, change correlation, LLM diagnosis, patching, validation and deployment in one auditable workflow. Its safety comes from the architecture, not the model.

A prototype of about 7.1 KLOC with 44 automated tests runs the complete loop. It was evaluated on four scenarios with known answers (three faulty releases and one harmless release); Table 7.1 summarises the results.

Table 7.1: Preliminary evaluation results

Measure	Heuristic (no LLM)	Local LLM (7B)
Detection correct (including no false positive)	4/4	4/4
True culprit ranked first by correlation	3/3	3/3
Diagnosis names the true culprit	3/3	2/3
AI patches passing validation	–	0/6
Reached the approval gate or correctly silent	4/4	3/4
Incorrect patches reaching the approval gate	0	0
Mean time per incident	≈ 5 s	≈ 100 s

Detection and correlation were accurate and deterministic. The local 7B model (qwen2.5-coder:7b) explained every failure and named the culprit in two of three cases, but none of its six patches passed validation; every wrong patch was stopped automatically, and the revert fallback produced validated candidates for two incidents. A small model is thus useful for diagnosis, and validation with a human gate keeps wrong patches out of production.

7.2 FUTURE WORK

Semester II will evaluate stronger models such as Claude on a larger benchmark, and add authentication, a container sandbox, secret redaction, GitHub Actions integration and automatic rollback.

APPENDIX A
PROBLEM STATEMENT FEASIBILITY ASSESSMENT

This appendix assesses feasibility using satisfiability analysis, complexity classes (P, NP-complete, NP-hard) and modern algebra.

A.1 ALGEBRAIC MODEL

Signatures as an equivalence relation. Define a relation ~ on the set of log events L by $l_1 \sim l_2 \Leftrightarrow s(l_1) = s(l_2)$. Being defined by equality of a function value, ~ is reflexive, symmetric and transitive, i.e. an equivalence relation. Its quotient set L/~ is exactly the set of distinct signatures, so deduplication is the mapping $L \to L/\sim$ and detection works on this much smaller set.

Incident life cycle as an automaton. The life cycle is a deterministic finite automaton $M = (Q, \Sigma, \delta, q_0, F)$, where Q is the set of twelve states (detected, analyzing, patching, validating, awaiting_approval, validation_failed, approved, deploying, resolved, deploy_failed, rejected, error), Σ the set of events (step outcomes and human actions), $\delta : Q \times \Sigma \to Q$ the partial transition function, q_0 = detected and F = { resolved, rejected }. Event sequences $\Sigma*$ form a free monoid under concatenation, and δ extends to $\delta* : Q \times \Sigma* \to Q$.

A.2 SATISFIABILITY ANALYSIS

The safety property is that production changes only through a validated, approved commit. Let Dep, V, Ap and M denote "the patch is deployed", "validation passed", "an approval was recorded" and "the approved SHA equals the patch SHA". The implementation enforces the constraint

$$\Phi = Dep \to (V \wedge Ap \wedge M) \qquad (A.1)$$

because the deployer checks all three conditions and the automaton has no edge into deploying except from approved. An unsafe deployment is described by $\Psi = Dep \wedge \neg(V \wedge Ap \wedge M)$. The conjunction $\Phi \wedge \Psi$ is unsatisfiable: any assignment with Dep = 1 must satisfy $V \wedge Ap \wedge M$ by Φ, which contradicts Ψ. Hence no execution can reach an unsafe deployment, while the success condition $Dep \wedge V \wedge Ap \wedge M$ remains satisfiable.

A.3 COMPLEXITY CLASSIFICATION

Table A.1 classifies the sub-problems.

Table A.1: Complexity class of each sub-problem

Sub-problem	Algorithm	Class
Normalisation and signatures	Regular-expression masking, hashing	P (linear)
Incident detection	Indexed window counts	P
Commit scoring and ranking	Weighted signals, sort	P
Minimum set of commits explaining all failures	Not attempted (Set Cover)	NP-complete
Bounded patch synthesis (finite choices)	Delegated to LLM proposals	NP-complete
Unbounded patch synthesis	Not attempted	Undecidable
Validation of a given patch (with timeouts)	Compile, test, replay	P
Approval and deployment checks	Constant number of comparisons	P

Bounded patch synthesis is NP-complete. Consider the decision problem: given a program with k Boolean "holes" and a test suite that runs in polynomial time, is there an assignment of the holes that makes all tests pass? It is in NP (an assignment is a certificate checked by running the tests) and NP-hard by reduction from SAT: for a CNF formula φ over $x_1 \dots x_k$, build the program `def f(): return φ(x₁, …, x_k)` with each x_i a hole and a single test asserting that f() returns True. The test passes for some assignment if and only if φ is satisfiable. Without a bound on the program, the problem is undecidable by Rice's theorem.

Minimal explanation is NP-complete. Let U be the set of failing signatures and let each commit c explain a subset $S_c \subseteq U$. Deciding whether k commits can explain all of U is exactly the Set Cover decision problem, which is NP-complete.

A.4 FEASIBILITY CONCLUSION

The full problem is NP-hard because it contains patch synthesis, but it is feasible in practice: the deterministic stages are in P, the hard search is delegated to a bounded heuristic (the LLM), and every proposal is verified in polynomial time. Safety does not depend on the heuristic, since $\Phi \wedge \Psi$ is unsatisfiable.

APPENDIX B
DETAILS OF THE PAPERS REFERRED

The papers that provided the seed ideas of this project are listed below in IEEE format, each with a summary of not more than four lines. The numbers match the References.

[2] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, pp. 81:1–81:45, 2021, doi: 10.1145/3483424.

Summary: Organises AIOps failure-management methods into prevention, detection, diagnosis and remediation. It provided the stage structure that AION connects into one loop.

[5] T. Ahmed *et al.*, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.

Summary: Evaluates GPT-3.x models on more than 40,000 incidents for recommending root causes and mitigation steps. It showed that LLMs can help, but only from incident text.

[6] Y. Chen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th European Conf. Computer Systems (EuroSys)*, 2024.

Summary: Performs automatic root-cause analysis of cloud incidents with LLMs. It inspired AION's LLM diagnosis stage, extended with code, deployment history and evidence citation.

[7] "Integrating large language models with cloud-native observability for automated root cause analysis and remediation," in *Proc. 3rd Int. Conf. Artificial Intelligence, Systems and Network Security*, 2025. doi: 10.1145/3797161.3797213.

Summary: Integrates LLMs with cloud-native observability for automated RCA and remediation and highlights its risks, supporting AION's human approval gate.

[11] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298. doi: 10.1145/3133956.3134015.

Summary: Learns normal log-key sequences with an LSTM and flags deviations as anomalies. It showed the value of log-based detection and its limit: detection without diagnosis.

[23] J. Śliwerski, T. Zimmermann, and A. Zeller, “When do changes induce fixes?” in *Proc. 2nd Int. Workshop on Mining Software Repositories (MSR)*, 2005, pp. 24–28.

Summary: Links version history to bug reports to find fix-inducing changes (the SZZ approach). AION applies the same idea online, using git blame on the failing lines at the deployed revision.

[24] C. Le Goues, M. Pradel, and A. Roychoudhury, “Automated program repair,” *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019, doi: 10.1145/3318162.

Summary: Reviews search-, semantics- and learning-based automated program repair and the role of tests as specifications, which AION relies on for validation.

[28] I. Bouzenia, P. Devanbu, and M. Pradel, “RepairAgent: An autonomous, LLM-based agent for program repair,” in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, 2025.

Summary: An autonomous LLM agent that gathers information, proposes fixes and validates them with tests. It inspired AION's bounded generate-validate-repair loop.

[31] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.

Summary: A benchmark of 2,294 real GitHub issues from 12 Python repositories for evaluating code-repair systems. It shaped AION's evaluation with known answers.

[32] U. Kulsum, H. Zhu, B. Xu, and M. d’Amorim, “A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback,” in *Proc. 1st ACM Int. Conf. AI-Powered Software (AIware)*, 2024.

Summary: Shows that reasoning and patch-validation feedback improve LLM-generated fixes. AION feeds validation failures back to the model in its repair loop.

[33] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Summary: Introduces retrieval-augmented generation, conditioning a model on retrieved passages. AION uses a curated evidence pack with identifiers that the model must cite.

APPENDIX C
PLAGIARISM REPORT

The plagiarism check report of this preliminary report, generated with the plagiarism-detection software provided by the institute, is attached on the following page.

REFERENCES

[1] Z. Chen *et al.*, "Towards intelligent incident management: Why we need it and how we make it," in *Proc. 28th ACM Joint Meeting on European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2020, pp. 1487–1497. doi: 10.1145/3368089.3417055.

[2] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, pp. 81:1–81:45, 2021, doi: 10.1145/3483424.

[3] L. Zhang, T. Jia, M. Jia, *et al.*, "A survey of AIOps in the era of large language models," *ACM Computing Surveys*, 2026, doi: 10.1145/3746635.

[4] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Computing Surveys*, vol. 54, no. 6, pp. 130:1–130:37, 2021, doi: 10.1145/3460345.

[5] T. Ahmed *et al.*, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.

[6] Y. Chen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th European Conf. Computer Systems (EuroSys)*, 2024.

[7] "Integrating large language models with cloud-native observability for automated root cause analysis and remediation," in *Proc. 3rd Int. Conf. Artificial Intelligence, Systems and Network Security*, 2025. doi: 10.1145/3797161.3797213.

[8] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE 34th Int. Symp. Software Reliability Engineering (ISSRE)*, 2023.

[9] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40. doi: 10.1109/ICWS.2017.13.

[10] Z. Jiang *et al.*, "LILAC: Log parsing using LLMs with adaptive parsing cache," in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024.

[11] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298. doi: 10.1145/3133956.3134015.

[12] W. Meng *et al.*, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745. doi: 10.24963/ijcai.2019/658.

[13] V.-H. Le and H. Zhang, “Log-based anomaly detection without log parsing,” in *Proc. 36th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2021, pp. 492–504. doi: 10.1109/ASE51524.2021.9678773.

[14] H. Guo, S. Yuan, and X. Wu, “LogBERT: Log anomaly detection via BERT.” arXiv:2103.04475, 2021.

[15] Y. Lee, J. Kim, and P. Kang, “LAnoBERT: System log anomaly detection based on BERT masked language model.” arXiv:2111.09564, 2023.

[16] Z. Ma, J. Yang, and T.-H. Chen, “LLM4Log: A systematic review of large language model-based log analysis.” arXiv:2604.16359, 2026.

[17] M. De la Cruz Cabello, T. P. Sales, and M. R. Machado, “Log anomaly detection in AIOps: A real-world implementation using large language models,” *Systems and Soft Computing*, vol. 8, p. 200475, 2026, doi: 10.1016/j.sasc.2026.200475.

[18] D. Roy *et al.*, “Exploring LLM-based agents for root cause analysis,” in *Companion Proc. 32nd ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024.

[19] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, “MicroRCA: Root cause localization of performance issues in microservices,” in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2020.

[20] M. Ma, J. Xu, Y. Wang, P. Chen, Z. Zhang, and P. Wang, “AutoMAP: Diagnose your microservice-based web applications automatically,” in *Proc. The Web Conference (WWW)*, 2020, pp. 246–258. doi: 10.1145/3366423.3380111.

[21] J. Soldani and A. Brogi, “Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey,” *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.

[22] T. Wittkopp, A. Acker, and O. Kao, “Progressing from anomaly detection to automated log labeling and pioneering root cause analysis,” in *Proc. AIOps Workshop, IEEE Int. Conf. Data Mining (ICDM)*, 2023.

[23] J. Śliwerski, T. Zimmermann, and A. Zeller, “When do changes induce fixes?” in *Proc. 2nd Int. Workshop on Mining Software Repositories (MSR)*, 2005, pp. 24–28.

[24] C. Le Goues, M. Pradel, and A. Roychoudhury, “Automated program repair,” *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019, doi: 10.1145/3318162.

[25] C. S. Xia and L. Zhang, “Less training, more repairing please: Revisiting automated program repair via zero-shot learning,” in *Proc. 30th ACM ESEC/FSE*, 2022. doi: 10.1145/3540250.3549101.

[26] C. S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.

[27] Q. Zhang *et al.*, "A systematic literature review on large language models for automated program repair." arXiv:2405.01466, 2024.

[28] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, 2025.

[29] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2024.

[30] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[31] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.

[32] U. Kulsum, H. Zhu, B. Xu, and M. d'Amorim, "A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback," in *Proc. 1st ACM Int. Conf. AI-Powered Software (AIware)*, 2024.

[33] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.